

VibeVideo Developer Guide

How to Add New Capabilities to the CLI, MCP & Engine Pipeline

Purpose: This guide provides a complete developer blueprint for adding new video, audio, transcription, or editing capabilities to VibeVideo. Follow the 5-layer pipeline to ensure smooth FAISS vector matching, MCP schema compliance, and robust engine execution.

1. The 5-Layer Capability Architecture

Whenever a user enters a command, it flows through 5 interconnected layers. Adding a capability requires updating each layer in order:

Layer	Component File	Role & Responsibility
1. Training / FAISS	documents/*.txt	Semantic vector embeddings. Associates natural phrasing with capability names.
2. Parsing	vibevideo.py	Extracts parameters (files, timestamps, multipliers) into a typed dict.
3. MCP Contract	mcp/registry.py	Declares the input/output schema and expected error codes for the action.
4. Tool Resolver	mcp/capability_resolver.py	Maps capability names to target engines (FFmpeg, Whisper, Audacity, etc.).
5. Engine Logic	engines/*_engine.py	Executes native CLI / model operations (FFmpeg commands, whisper transcribe, etc.).

2. Step-by-Step Implementation Workflow

Follow these 5 steps in order whenever introducing a new single capability (illustrated with split_video):

Step 1: Add Natural Language Training Data (FAISS)

Open the relevant document file in documents/ (e.g. documents/ffmpeg.txt) and add a new section delimited by \$\$\$|||. The first line is the **action name**, followed by natural phrasing examples:

```
$$$|||
split_video
split video at 01:00
split videoxyz.mp4 at 00:07
split this video in two parts at 10 seconds
divide video into two parts at 01:30
cut video in two at 00:45
```

Step 2: Register Schema in MCP Registry

Open mcp/registry.py and register the capability in MCP_REGISTRY. Specify required inputs, outputs, and errors:

```
MCP_REGISTRY = {  
    ...  
    "split_video": {  
        "action": "split_video",  
        "input": ["input_files", "start_time"],  
        "output": ["output_file"],  
        "errors": ["file_not_found", "invalid_timestamp", "split_failed"]  
    },  
}
```

Step 3: Map Capability in Resolver

Open `mcp/capability_resolver.py` and add an entry to `CAPABILITY_TOOL_MAP` connecting the action to its engine:

```
CAPABILITY_TOOL_MAP = {  
    ...  
    "split_video": {"tool": "ffmpeg", "implementation": "split_video"},  
}
```

Step 4: Implement Logic in Target Engine

In `engines/ffmpeg_engine.py` (inside `execute_ffmpeg`), implement the handler. Read inputs, validate parameters, run FFmpeg, and return the generated path(s):

```
elif implementation == "split_video":
    if not input_files: raise ValueError("No input file specified.")
    input_file = input_files[0]
    split_time = inp.get("start_time")
    split_sec = _parse_time_to_seconds(split_time)
    total_dur = get_video_duration(input_file, ffmpeg_path)
    if total_dur is not None and split_sec >= total_dur:
        raise ValueError("Split timestamp exceeds video duration.")
    base, ext = os.path.splitext(input_file)
    p1, p2 = f"{base}_part1{ext}", f"{base}_part2{ext}"
    # Part 1: from 00:00 to split_sec
    subprocess.run(f'"{ffmpeg_path}" -y -i "{input_file}" -t {split_sec} -c:v libx264 -c:a aac "{p1}"', shell=True)
    # Part 2: from split_sec to end
    subprocess.run(f'"{ffmpeg_path}" -y -ss {split_sec} -i "{input_file}" -c:v libx264 -c:a aac "{p2}"', shell=True)
    return [p1, p2]
```

Step 5: Verify Query Parsing in `vibevideo.py`

Ensure `parse_parameters()` in `vibevideo.py` extracts the relevant keywords/timestamps (e.g. at 00:07 into `params["start_time"]`). The CLI automatically handles returned lists and displays all new output files.

3. Concrete Test Case Walkthrough: Split Video Example

Here is the exact real-world test case conducted on `Mohini_Dey_Live_Show.mp4` demonstrating the diagnostic failure before vs. the successful execution after adding the capability:

A. Before Implementation (Diagnostic Test)

Command: `split Mohini_Dey_Live_Show.mp4 at 00:07`

[Tier 1] No confident multicommand match (best='split_dual_subtitle', distance=1.132); fallback to single.

Capability: `clip_by_semantic` (*FAISS misclassified due to missing split_video training*)

Parsed Parameters: `{'filename': 'Mohini_Dey_Live_Show.mp4', 'start_time': '00:07', ...}`

MCP Instruction: `{'action': 'clip_by_semantic', 'input': {'search_query': None}, ...}`

Selected Tool: `{'tool': 'whisper', 'implementation': 'clip_by_semantic'}`

Result: Error running '`clip_by_semantic`': `clip_by_semantic` requires a search_query.

B. After Implementation Across Steps 1 to 5

Command: split Mohini_Dey_Live_Show.mp4 at 00:07

[Tier 1] No multicommand match; falling back to single command.

Capability: split_video (*Matched via documents/ffmpeg.txt*)

Parsed Parameters: {'input_files': ['Mohini_Dey_Live_Show.mp4'], 'start_time': '00:07', ...}

MCP Instruction: {'action': 'split_video', 'input': {'input_files': ['Mohini_Dey_Live_Show.mp4'], 'start_time': '00:07'}}

Selected Tool: {'tool': 'ffmpeg', 'implementation': 'split_video'}

Executing Part 1: ffmpeg -y -i Mohini_Dey_Live_Show.mp4 -t 7.0 -c:v libx264 -c:a aac

Mohini_Dey_Live_Show_part1.mp4

Executing Part 2: ffmpeg -y -ss 7.0 -i Mohini_Dey_Live_Show.mp4 -c:v libx264 -c:a aac

Mohini_Dey_Live_Show_part2.mp4

New outputs:

[1] sample_media/Mohini_Dey_Live_Show_part1.mp4

[2] sample_media/Mohini_Dey_Live_Show_part2.mp4

4. Developer Checklist & Best Practices

- **Exact Naming Match:** The capability key in documents/*.txt (first line), mcp/registry.py, and mcp/capability_resolver.py must match character-for-character.
- **Return Format:** Engine functions should return either a string filepath or a list of generated filepaths ([path1, path2]).
- **Re-encoding vs Stream Copy:** When trimming or splitting at timestamps, prefer re-encoding (-c:v libx264 -c:a aac) over stream copy (-c copy) to prevent frozen frame or keyframe misalignment issues.
- **Error Raising:** Raise explicit ValueError or FileNotFoundError with descriptive messages inside engine handlers so the CLI reports actionable errors to the user.
- **Re-indexing:** FAISS vector indexes are rebuilt dynamically on every CLI startup from documents/ and multicommand_documents/. Simply restart python vibevideo.py to reload.